

Chapter 4 — Complexity Analysis

Course notes summary · asymptotic runtime, big-O, counting steps, logs, loop dependencies, Θ/Ω , space complexity, profiling

How to read these notes: ★ **NOT TESTED** = purple dashed box, safe to skim (still included for completeness). Blue box = definition/key rule. Green box = worked example. Yellow box = remark / common pitfall.

Chapter status: this chapter is a core foundation of the class — know everything except where marked. Only **Section 4.8 (limits & L'Hôpital)** is flagged as not-required (no calculus needed).

4.1 Asymptotic Runtime

Why pure runtime is a bad metric

- Naive idea: run algorithm A and algorithm B, whichever finishes first is "more efficient."
- Broken because of **confounding factors**:
 - Different machines → a faster CPU can make the worse algorithm look better.
 - Same machine still not enough → background processes, other programs running in parallel.
 - Compiler and compiler flags used.
- Conclusion: **pure runtime is an imperfect metric**; it measures the environment as much as the algorithm.

The fix: measure scaling with input size

- Instead of "How long does this take?" ask "**How does runtime scale as input size grows?**"
 - Double the input → does runtime double? quadruple? stay the same?
- Given two algorithms doing the same task, the one whose runtime **grows slower** as input grows is the more time-efficient one.
- Why this is better: for **large enough input sizes**, the *rate of growth* is independent of most external factors (CPU speed, flags, background processes).

Definition — Asymptotic Runtime

Describes **how the runtime of an algorithm grows as the size of its input grows**. Used to measure and compare time-efficiency of algorithms.

Asymptotic runtime ≠ actual runtime

- "Program A finishes in 2.81 seconds" → that is **runtime** (a unit of time).
- "Program A's runtime doubles when input doubles, triples when input triples" → that is a **linear relationship**, i.e. its **asymptotic runtime**.
- Asymptotic runtime is expressed with **big-O notation**, not seconds.

4.2 Complexity Classes and Big-O Notation

Choosing what "one unit of input" means

- Depends on what the algorithm does — pick a unit related to the algorithm's functionality.
- Sorting an array of integers → input size = number of integers in the array.
- Graph with V vertices and E edges:
 - Algorithm only touches vertices → define input size in terms of V .
 - Algorithm works on both → define it in terms of **both V and E** .
 - More variables included → more information out of the analysis (e.g. which variable dominates performance).
- When measuring experimentally, **use large input sizes**. Small inputs get obfuscated by variance/noise and hide the true relationship.

Example 4.1 — measuring `sum_array()` experimentally

```
int32_t sum_array(int32_t arr[], size_t size) {
    int32_t total = 0;
    for (size_t i = 0; i < size; ++i) {
        total += arr[i];
    } // for i
    return total;
} // sum_array()
```

Input size = size of the array. Measured data:

Input Size	10000	20000	30000	40000	50000	100000	250000
Runtime (ms)	2.5	2.9	3.2	3.5	3.9	5.7	~11.9

Graphing gives a straight line → a **linear relationship** between input size and runtime.

Definition — Time Complexity

A way to express the **relationship between runtime and input size**.

- Linear relationship → **linear time complexity**
- Quadratic relationship → **quadratic time complexity**
- Runtime stays flat regardless of input → **constant time complexity**

Big-O notation

- Expresses time complexity mathematically instead of with words: form **$O(f(...))$** , where $f(...)$ is a function of the algorithm's input size variables.

3 steps to get the big-O from a runtime equation

1. Express the runtime as a **function of the input size variables**.
 2. Identify the **fastest growing term(s)** and drop all lower order terms.
 3. Drop the **coefficients** of the fastest growing terms.
- What's left is $f(n)$.

Example 4.2 — big-O of sum_array()

1. Line of best fit through the data points:

$$T(n) = (4 \times 10^{-5})n + 2$$

2. Fastest growing term: the constant 2 never grows; $(4 \times 10^{-5})n$ grows linearly $\rightarrow$ keep $(4 \times 10^{-5})n$.
 - Usually the fastest growing term is the one with the **largest exponent**. E.g. for $T(n) = 5n^3 + 2n^7 + 16n^6 + 9n^2 + 55n^4 + 23$, the fastest growing term is $2n^7$.
3. Drop the coefficient $4 \times 10^{-5} \rightarrow$ left with n .

sum_array() is $O(n)$.

Example 4.3 — return_zero()

```
int32_t return_zero(int32_t arr[], size_t size) {  
    int32_t zero = 0;  
    return zero;  
} // return_zero()
```

Runtime is 1.7 ms for input sizes 10000, 25000, 50000, 75000, 100000 — a flat line.

1. $T(n) = 1.7$
2. Only one term, so it is by default the fastest growing term.
3. 1.7 can be written as $1.7n^0$; drop the coefficient $1.7 \rightarrow n^0 = 1$.

return_zero() is $O(1)$.

Why we're allowed to drop coefficients and lower order terms

- Time complexity only describes how runtime **scales**, not the actual runtime. Coefficients and lower order terms are not responsible for the rate of growth.
- $T(n) = 0.5n^2 + 13n + 5$ and $T(n) = 14n^2 + 3n + 9$ both grow quadratically $\rightarrow$ both meaningfully expressed as $O(n^2)$.

Remark — dropped does not mean unimportant

- $T(n) = 999n^2$ is **slower** than $T(n) = 0.01n^3$ for most reasonable input sizes, even though it has the better complexity class.
- **A better time complexity does not guarantee a faster runtime.** Complexity only indicates how runtime scales with input size.

Common complexity classes (in order of rate of growth)

Name of Complexity Class	Big-O Notation
Constant	$O(1)$
Logarithmic	$O(\log(n))$
Linear	$O(n)$
Loglinear / Linearithmic	$O(n \log(n))$
Quadratic	$O(n^2)$
Cubic	$O(n^3)$
Polynomial (includes quadratic and cubic)	$O(n^d)$, d a constant
Exponential	$O(c^n)$, c a constant
Factorial	$O(n!)$
Doubly Exponential	$O(2^{2^n})$

Visualized: constant and logarithmic stay nearly flat; linear is a straight diagonal; linearithmic curves up slightly faster; quadratic, exponential, factorial and doubly exponential shoot up almost vertically at small input sizes.

Why experimentation is not enough

- Not always feasible (you cannot run code on an exam).
- Even with data: hard to fit an equation, hard to know how large inputs must be to avoid noise.
- Multiple input variables (V and E) are hard to separate experimentally.
- **Good news:** you don't need a computer — just reading the code is usually enough, by counting **steps** instead of measuring time.

4.3 Measuring Complexity by Counting Steps

- Count the **number of steps** an algorithm takes and express it as a function of input size.
- Works because: as long as a single step takes constant time (independent of input size), the step-count function has the **same order of growth** as the runtime function.
 - Linear growth in step count $\rightarrow$ linear growth in runtime.
 - Quadratic blowup in steps $\rightarrow$ quadratic blowup in runtime.

What counts as one step (constant-time operations)

1. variable assignment (`a = 5`)
2. arithmetic operation (`a + b`)
3. comparison operation (`a < b`)
4. array indexing (`arr[i]`)
5. pointer reference (`*ptr`)
6. function call
7. function return

Footnote: for some algorithms input size is measured in *bits/digits* of an integer, in which case calling arithmetic "constant time" is an oversimplification — not a concern for this class.

Procedure (same shape as section 4.2)

1. Express the **step count** as a function of the input size variables.
2. Identify the fastest growing terms, drop lower order terms.
3. Drop the coefficients.

Example 4.4 — exact step count of `sum_array()`

Line-by-line (input size `n = size`):

- **Line 1** (function header): ignored — the work of calling the function is charged to the **caller**, not to `sum_array()`.
- **Line 2** `int32_t total = 0;` → 1 assignment = 1 step.
- **Line 3** `for (size_t i = 0; i < size; ++i)` → 1 init assignment + $(n+1)$ comparisons (one per loop body run, plus the final failing check) + n increments.
- **Line 4** `total += arr[i];` → 1 step, run n times = n .
- **Line 6** `return total;` → 1 step.

Total: $1 + [1 + (n+1) + n] + n + 1 = \mathbf{S(n) = 3n + 4}$ → fastest growing term $3n$ → drop coefficient → **$O(n)$** .

Key insight — exact step counts don't matter

- `+=` is really two steps (add then assign); `++i` is also two (increment then assign). So $3n+4$ is technically inaccurate; the true count is **$5n + 4$** .
- Both are still **linear**. Whether a line is 1, 2, 3 or 999 steps does not change the complexity class, as long as it is a constant.
- → Don't compute exact step counts. Just assign each line its **asymptotic complexity**.

Calculating time complexity of a function by counting steps

1. Start at the first line of the function (**not** the header). Walk line by line and assign each line a value:
 - **a.** If the line is a **loop definition** (`for(...)`, `while(...)`), assign it the **number of times the loop executes** (ignore coefficients / lower order terms).
 - **b.** Otherwise, assign it the **time complexity required to execute that line**.
2. Identify the loops. For a **non-nested loop**: add all complexities in the loop body, then **multiply** by the number of times the loop runs.
3. For **nested loops**: start at the **innermost** loop and work outwards, condensing one loop at a time using step 2.
4. Once all loops are condensed, **add** the remaining complexities. Drop coefficients and lower order terms.

Example 4.5 — `sum_array()` by the procedure

```
int32_t sum_array(int32_t arr[], size_t size) {
    int32_t total = 0;
    for (size_t i = 0; i < size; ++i) {
        total += arr[i];
    } // for i
    return total;
} // sum_array()
```

$O(1)$
 $O(n)$ ← loop runs n times
 $O(1)$

 $O(1)$

Condense the loop: $O(n)$ iterations $\times$ $O(1)$ body = **$O(n)$** .

Add the rest: $T(n) = O(1) + O(n) + O(1) = \mathbf{O(n)}$.

Example 4.6 — nested loops, foo()

```
1 void foo(int32_t arr[], size_t size) {
2   int32_t num = static_cast<int32_t>(size);
3   while (num > 0) {
4     for (size_t j = 0; j < size; ++j) {
5       for (size_t k = size - 1; k > (size / 2); --k) {
6         std::cout << arr[k] << '\n';
7         sum_array(arr, size);
8       } // for k
9     } // for j
10    --num;
11  } // while
12  for (size_t i = 0; i < size; ++i) {
13    std::cout << arr[i] << '\n';
14  } // for i
15  return;
16 } // foo()
```

Per-line assignment:

- L2 assignment $\rightarrow O(1)$
- L3 while: num starts at size and is decremented each pass $\rightarrow$ runs n times $\rightarrow O(n)$
- L4 for: 0 $\rightarrow$ size by 1 $\rightarrow O(n)$
- L5 for: n-1 down to n/2 $\rightarrow$ roughly n/2 times $\rightarrow O(n)$
- L6 print with array index, independent of input size $\rightarrow O(1)$
- L7 sum_array() is linear $\rightarrow O(n)$
- L10 $O(1)$, L12 loop $O(n)$, L13 $O(1)$, L15 $O(1)$

Condense from the inside out:

1. Body of L5 loop = $O(1) + O(n) = O(n)$. Loop runs $O(n)$ times $\rightarrow$ L5 loop = $O(n) \times O(n) = O(n^2)$.
2. L4 loop runs $O(n)$ times over an $O(n^2)$ body $\rightarrow O(n^3)$.
3. L3 while body = $O(n^3) + O(1) = O(n^3)$, run $O(n)$ times $\rightarrow O(n^4)$.
4. L12 loop: $O(n) \times O(1) = O(n)$.

Final: $O(1) + O(n^4) + O(n) + O(1) = O(n^4)$.

The triple-nested loop is the **bottleneck** that governs the growth rate of the whole function.

4.4 Logarithmic Runtime

4.4.1 Analyzing logarithmic runtime

- Ordinary increment loop for $(i = 1; i < \text{input_size}; ++i) \rightarrow$ runs n times.
- **Multiplication** loop for $(i = 1; i < \text{input_size}; i *= 2)$: i goes 1, 2, 4, 8, 16... The number of iterations k satisfies

$$2^k = n \rightarrow k = \log_2(n)$$

So the loop runs **$O(\log_2(n))$** times.

- n = 64: loop runs for i = 1, 2, 4, 8, 16, 32 $\rightarrow$ 6 times, and $\log_2(64) = 6$. For n = 65 it runs 7 times, still on the order of $\log_2(n)$.
- **Division** loop for $(i = \text{input_size}; i > 1; i /= 2)$: how many times can n be halved before hitting 1?

$$n(1/2)^k = 1 \rightarrow k = \log_2(n)$$

Same answer as multiplication.

Rules of thumb for spotting logs

- If **multiplication or division** appears in a loop update, there is a good chance a logarithmic term is in play.
- If the **search space is cut in half** (or by any constant factor) each iteration, a log term is likely involved.

Real-world example — phone book

n pages, looking for a last name.

- Check every page one by one $\rightarrow O(n)$.
- Book is alphabetical $\rightarrow$ looking for "P", ignore the first half entirely; flip to the middle of the remaining half and repeat. Half the remaining pages are eliminated each step (same as $i /= 2$) $\rightarrow$ at most **$O(\log_2(n))$** pages checked.

Example 4.7 — doubling loop

```
int32_t foo(int32_t arr[], size_t size) {
  int32_t total = 0;           // O(1)
  for (size_t i = 1; i < size; i *= 2) { // O(log2(n))
    total += arr[i];           // O(1)
  } // for i
  return total;                // O(1)
} // foo()
```

Loop = $O(\log_2(n)) \times O(1) = O(\log_2(n))$. Total = $O(1) + O(\log_2(n)) + O(1) = O(\log_2(n))$.

Example 4.8 — triple nested logs

```
void foo(int32_t n) {
    for (int32_t i = 1; i < n; ++i) {           // O(n)
        for (int32_t j = 1; j < log2(n); j *= 2) { // O(log2(log2(n)))
            for (int32_t k = 1; k < n; k *= 2) { // O(log2(n))
                std::cout << "Potato!\n";       // O(1)
            } } } }
```

- L3 bound is $\log_2(n)$ with a doubling counter $\rightarrow 2^k = \log_2(n) \rightarrow k = \log_2(\log_2(n))$.
- Innermost (L4): $O(\log_2(n)) \times O(1) = O(\log_2(n))$.
- L3: $O(\log_2(\log_2(n))) \times O(\log_2(n))$.
- L2: $\times O(n)$.

$O(n \times \log_2(\log_2(n)) \times \log_2(n))$

Example 4.9 — power_sum()

```
int32_t power_sum(int32_t n) {
    int32_t total = 0;           // O(1)
    int32_t i = 1;              // O(1)
    while (i < n) {              // O(log2(n)) -- i doubles each pass
        total += i;              // O(1)
        i *= 2;                 // O(1)
    } // while
    return total;                // O(1)
} // power_sum()
```

Loop = $O(\log_2(n))$ iterations $\times O(1)$ work = $O(\log_2(n))$. Overall **$O(\log_2(n))$** .

4.4.2 Does the base of a logarithm matter?

- **No.** $O(\log_2(n))$ and $O(\log_3(n))$ are the same complexity class, written $O(\log(n))$.
- Reason — the **change of base formula**:

$$\log_b(n) = \log_a(n) / \log_a(b)$$

Changing base just multiplies by the constant $1/\log_a(b)$, which is dropped as a coefficient.

Example 4.10 — $\log_2(n)$ is both $O(\log_2(n))$ and $O(\log_3(n))$

$f(n)$ is by definition $O(f(n))$, so $\log_2(n)$ is $O(\log_2(n))$. By change of base:

$$\log_2(n) = \log_3(n) / \log_3(2)$$

$1/\log_3(2) \approx 1.58$ is a constant coefficient, so it drops $\rightarrow \log_2(n) = O(\log_3(n))$. Same for any base (2, 3, or 999): the difference is only a constant factor that does not grow with n .

This does NOT apply to exponents

The base of an **exponent** matters. $O(2^n)$ and $O(8^n)$ are **different** classes, because $8^n = 2^n \times 2^{2n}$, and 2^{2n} is not a constant factor.

4.4.3 Logarithmic and power identities

Log Identities	Power Identities
1) $\log_a(xy) = \log_a(x) + \log_a(y)$	1) $a^{n+m} = a^n a^m$
2) $\log_a(x/y) = \log_a(x) - \log_a(y)$	2) $a^{n-m} = a^n / a^m$
3) $\log_a(x^r) = r \log_a(x)$	3) $(a^n)^m = a^{nm}$
4) $\log_a(1/x) = -\log_a(x)$	4) $a^{-n} = 1/a^n$
5) $\log_a(x) = \log(x) / \log(a)$	
6) $\log_a(a) = 1$	
7) $\log_a(1) = 0$	

Example 4.11 — simplifying with identities

$$T(n) = \log(n) + \log(n^{17} \times 3^{n+5})$$

- Log identity 1: $= \log(n) + \log(n^{17}) + \log(3^{n+5})$
- Log identity 3: $\log(n^{17}) = 17 \log(n)$
- Power identity 1: $\log(3^{n+5}) = \log(3^n \times 3^5) = \log(3^n) + \log(3^5)$
- Log identity 3 again: $\log(3^n) = n \log(3)$

$$T(n) = 18 \log(n) + n \log(3) + \log(243)$$

$\log(3)$ is a constant coefficient, so the highest order term is n . **$T(n) = O(n)$** .

Example 4.12 — two variables, m and n

```
int32_t foo(int32_t m, int32_t n) {
    int32_t c = 0; // 0(1)
    while (m > n) { // 0(log(m/n))
        m = m / 2; // 0(1)
        for (int32_t i = 1; i < n; i *= 3) { // 0(log(n))
            ++c; // 0(1)
        } // for i
    } // while
    return c; // 0(1)
} // foo()
```

- Inner for: i tripled each pass $\rightarrow \log_3(n)$ times $\rightarrow O(\log(n))$.
- Outer while: if the condition were $m > 1$, m halving would give $\log_2(m)$ iterations. But we stop at n, so we **lose** $\log_2(n)$ iterations:
$$\text{iterations} = \log_2(m) - \log_2(n) = \log_2(m/n)$$
- *Concrete check:* $m = 256$, stop at $1 \rightarrow 256, 128, 64, 32, 16, 8, 4, 2 = \log_2(256) = 8$ iterations. With $n = 32$, only $256, 128, 64$ run = 3 iterations = $\log_2(256) - \log_2(32) = 8 - 5$.

Overall: $O(\log(m/n) \times \log(n))$

Faster route: the value of m at iteration k is $m(1/2)^k$; the loop stops when $m(1/2)^k = n$, so $k = \log_2(m/n)$.

4.5 Loop Dependencies

What is a loop dependency?

When the **terminating condition of an inner loop depends on a value that changes in the outer loop**, the inner loop runs a different number of steps each pass. You **cannot** analyze the loops independently and multiply.

Approach: consider the whole dependency together and count the total number of steps that occur across all iterations of the outermost loop in the dependency (usually producing a summation / geometric series).

The trap

```
for (size_t i = size; i > 1; i /= 2) {
    for (size_t j = 0; j <= i; ++j) { ++count; }
}
```

Tempting (and **wrong**) answer: outer is $O(\log n)$, inner is $O(n)$, so $O(n \log n)$. The inner bound is **i**, which shrinks with the outer loop $\rightarrow$ must analyze together.

Sum of a geometric series (useful for these problems)

For n terms, first term a, common ratio r:

$$S_n = a(1 - r^n) / (1 - r), \quad r \neq 1$$

Example 4.13 — halving outer, linear inner

```
int32_t foo(int32_t arr[], size_t size) {
    int32_t count = 0;
    for (size_t i = size; i > 1; i /= 2) {
        for (size_t j = 0; j <= i; ++j) { ++count; }
    } // for i
    return count;
} // foo()
```

- 1st outer pass: $i = n \rightarrow$ inner runs n times. 2nd: $i = n/2 \rightarrow$ inner runs $n/2$. 3rd: $n/4$. And so on.
- Outer runs $\log_2(n)$ times total.

$$T(n) = n + n/2 + n/4 + n/8 + \dots = n \cdot \sum_{k=0}^{\log_2(n)} (1/2)^k$$

Geometric series ($a = 1$, $r = 1/2$, $\log_2(n)+1$ terms):

$$T(n) = n[2 - 2(1/2)^{\log_2(n)+1}] \approx 2n$$

The bracket is a constant approaching 2 as n grows. **Nested loop is $O(n)$** , and since lines 2 and 8 are constant, **foo() is $O(n)$** .

Intuition: $n(1 + 1/2 + 1/4 + 1/8 + \dots) \approx 2n$.

Example 4.14 — tripling middle loop

```
void foo(int32_t n) {
    for (int32_t i = 0; i < n; ++i) { // runs n times
        for (int32_t j = 1; j < n; j *= 3) { // dependency with k loop
            for (int32_t k = 0; k < j; ++k) { // runs j times
                std::cout << "Potato!\n";
            } } }
}
```

Lines 3 and 4 are dependent (k's bound is j). Inner runs 1, then 3, then 9, then 27... up to n:

$$1 + 3 + 9 + \dots + n = \sum_{k=0}^{\log_3(n)} 3^k = (3n - 1)/2 = O(n)$$

So lines 3-7 take $O(n)$, and the L2 loop runs it n times $\rightarrow O(n) \times O(n) = O(n^2)$.

4.6 Big-O, Big-Θ, and Big-Ω

Formal definitions

- **Big-O (asymptotic upper bound):** $f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq c g(n)$ for all $n \geq n_0$.
- **Big-Ω (asymptotic lower bound):** $f(n)$ is $\Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq c g(n)$ for all $n \geq n_0$.
- **Big-Θ (asymptotic tight bound):** $f(n)$ is $\Theta(g(n))$ if there exist positive constants c_1, c_2, n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

Notation	Bound	Analogy	Typical use
$O(g(n))$	upper	$\leq$	worst-case: growth is no worse than $g(n)$
$\Omega(g(n))$	lower	$\geq$	best-case: growth is no better than $g(n)$
$\Theta(g(n))$	tight	$=$	same order of growth; the tightest bound
$o(g(n))$	strict upper (not tight)	$<$	$f(n) < c g(n)$ for all $n \geq n_0$
$\omega(g(n))$	strict lower (not tight)	$>$	$f(n) > c g(n)$ for all $n \geq n_0$

Bounds are not always tight

- Asked "how many students are in EECS 281?" — "less than one million" and "more than five" are both **true** but **useless**.
- Same idea: a constant time algorithm is technically $O(2^n)$, and an exponential algorithm is technically $\Omega(1)$. Correct, but meaningless.
- That is why we use **big-Θ** when we want the tightest bound. In this course we typically use the tightest possible bound (earlier examples could all have been written with Θ).
- By definition: if $T(n)$ is both $O(g(n))$ **and** $\Omega(g(n))$, it is also $\Theta(g(n))$ — just like $\leq$ and $\geq$ together implying $=$.

Complexity rules

Transitivity

1. $f = \Theta(g)$ AND $g = \Theta(h) \Rightarrow f = \Theta(h)$ (same rate as g , g same rate as $h \rightarrow f$ same rate as h)
2. $f = O(g)$ AND $g = O(h) \Rightarrow f = O(h)$ (grows slower-or-equal chains)
3. $f = \Omega(g)$ AND $g = \Omega(h) \Rightarrow f = \Omega(h)$ (grows faster-or-equal chains)

Reflexivity (a function is big-O of itself)

1. $f(n) = \Theta(f(n))$
2. $f(n) = O(f(n))$
3. $f(n) = \Omega(f(n))$

Symmetry / transpose symmetry

1. $f(n) = \Theta(g(n))$ **if and only if** $g(n) = \Theta(f(n))$
2. $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$

Example 4.15 — prove $\log(n!) = \Theta(n \log(n))$

Strategy: show it is both $O(n \log n)$ and $\Omega(n \log n)$.

Upper bound. Expand with log identity 1:

$$\log(n!) = \log(n) + \log(n-1) + \dots + \log(2) + \log(1)$$

Every term is $\leq \log(n)$, and there are n terms:

$$\leq \log(n) + \log(n) + \dots + \log(n) = n \log(n) \Rightarrow \log(n!) = O(n \log n)$$

Lower bound. Throw away the smaller half of the terms (all $\log(k)$ with $k < n/2$), then replace each remaining term with the smallest one, $\log(n/2)$. There are $n/2$ remaining terms:

$$\log(n!) \geq (n/2) \log(n/2)$$

Now show $(n/2) \log(n/2) = \Theta(n \log n)$:

$$(n/2) \log(n/2) = (n/2)(\log n - \log 2) = \frac{1}{2}(n \log n) - (\log 2/2)(n) = \Theta(n \log n)$$

So $\log(n!) = \Omega(n \log n)$. Both bounds hold $\rightarrow$ **$\log(n!) = \Theta(n \log(n))$** .

This result shows up repeatedly (e.g. sorting lower bounds, chapter 5 recurrences) — worth remembering.

4.7 Complexity Metrics: Best / Worst / Average Case

The three metrics (for a fixed input size n)

- **Best-case:** complexity in the very best case — the **lower bound** on running time; behavior under optimal conditions.
- **Worst-case:** complexity in the very worst case — the **upper bound**; the worst growth over all possible inputs. Most commonly reported metric.
 - Caveat: it does not account for **how often** the worst case actually happens. If the worst case is rare, it may misrepresent true performance (handled by **amortized complexity**, chapter 12).
- **Average-case:** the average complexity over all possible inputs of size n .

Input size is fixed for all three

Only the **contents** of the input may change, not its size.

- **Wrong:** "the best case for a sorting algorithm is when the array has size 1" — that changes the input size.
- **Right:** "the best case is when the input is already sorted" — contents changed, size unchanged.

Example 4.16 — linear search for a value known to be in an array of n integers

- **Best case:** target is the first element $\rightarrow$ one check $\rightarrow \Theta(1)$.
- **Worst case:** target is the last element (or absent) $\rightarrow$ all n elements checked $\rightarrow \Theta(n)$.
- **Average case** (assume the target is uniformly likely to be at any index): 1 comparison if it's 1st, 2 if 2nd, ..., n if n^{th} .

$$\text{avg} = (1 + 2 + \dots + n)/n = [n(n+1)/2]/n = (n+1)/2 = \Theta(n)$$

Formal version (stats background): let X = number of comparisons before finding the target; $P(X = x) = 1/n$ for a uniform distribution. Then $E[X] = \sum_{x=1}^n x \cdot P(x) = \sum_{x=1}^n x/n = (n+1)/2 = \Theta(n)$.

In this class you will usually be **given** the average-case complexities of important algorithms rather than deriving them — but you should understand what an average-case analysis measures.

4.8 Asymptotic Analysis with Limits and L'Hôpital's Rule ★ NOT TESTED

★ **No calculus is required for this class. This section is only an alternative method for finding a complexity class.**

Some functions have no relationship at all. It is possible for two functions to have neither a big- O , big- Θ , nor big- Ω relationship. Example: $f(n) = \sin(n)$, $g(n) = \cos(n)$. There is no constant you can multiply $\cos(n)$ by to make it always grow faster or slower than $\sin(n)$ beyond some n_0 . (Trigonometric runtimes are rare in practice.)

Limit condition (sufficient but NOT necessary): if

$$\lim_{n \rightarrow \infty} (f(n) / g(n)) = d < \infty$$

then $f(n) = O(g(n))$. Because it is only sufficient, it **cannot** be used to prove that $f(n) \neq O(g(n))$.

L'Hôpital's Rule: if f and g are differentiable and $\lim_{n \rightarrow c} (f(n)/g(n)) = 0/0$ or ∞/∞ , then

$$\lim_{n \rightarrow c} (f(n)/g(n)) = \lim_{n \rightarrow c} (f'(n)/g'(n))$$

Example 4.17 — prove $\log_2(n)$ is $O(n)$. $f = \log_2(n)$, $g = n$. The limit is ∞/∞ , so apply L'Hôpital: derivative of $\log_2(n)$ is $1/(n \ln 2)$, derivative of n is 1.

$$\lim_{n \rightarrow \infty} 1/(n \ln(2)) = 0 < \infty$$

0 is a constant less than ∞ , so $\log_2(n) = O(n)$.

Example 4.18 — prove $\ln(n)$ is $O(\ln(n/281))$. Simplify $g = \ln(n/281) = \ln(n) - \ln(281)$. The limit is ∞/∞ ; derivative of $\ln(n)$ is $1/n$, derivative of the constant $\ln(281)$ is 0:

$$\lim_{n \rightarrow \infty} (1/n) / (1/n) = 1 < \infty$$

So $\ln(n) = O(\ln(n/281))$.

4.9 Space Complexity and Auxiliary Space

- Big- O is not only for runtime — it can express the growth of **any computational resource** with respect to input size.

Definitions

- **Space complexity** (a.k.a. memory complexity): the **total** amount of memory used by an algorithm to solve a problem, with respect to input size. **Includes the memory holding the input values themselves.**
- **Auxiliary space:** the **temporary/additional** space allocated by the algorithm, with respect to input size. **Excludes** the memory used by the input values.

In this class we mostly deal with auxiliary space.

- $\Theta(1)$ auxiliary space $\rightarrow$ extra memory does not depend on input size.
- $\Theta(n)$ auxiliary space $\rightarrow$ extra memory grows linearly with input.
- $\Theta(n^2)$ auxiliary space $\rightarrow$ quadratic increase in extra memory.

Sources of memory that are easy to miss

- **Pass by value:** a parameter passed by value (not by reference) makes a **copy**, which counts toward auxiliary space.
- **Recursion:** each recursive call may allocate a **stack frame**. If the number of calls depends on input size, so does the number of frames.
- $\rightarrow$ A function that never explicitly allocates memory can still have auxiliary space worse than $\Theta(1)$. (Covered in depth in chapter 5.)

Example 4.19 — auxiliary space of `foo()`

```
1 int32_t foo(int32_t arr[], size_t size) {
2   int32_t* bar = new int32_t[size];    //  $\Theta(n)$ 
3   int32_t counter = 0;                 //  $\Theta(1)$ 
4   for (size_t i = 0; i < size; ++i) {   //  $\Theta(1)$ 
5     bar[i] = arr[size - 1 - i];
6   } // for i
7   for (size_t j = 0; j < size; ++j) {   //  $\Theta(1)$ 
8     counter += (bar[j] + arr[j]);
9   } // for j
10  delete[] bar;
11  return counter;
12 }
```

- Allocations happen on lines 2, 3, 4 and 7.
- Lines 3, 4, 7 each instantiate a single integer $\rightarrow$ independent of input size $\rightarrow \Theta(1)$ each.
- Line 2 allocates an array sized by the input $\rightarrow$ grows linearly $\rightarrow \Theta(n)$.

Drop lower order terms: **overall auxiliary space = $\Theta(n)$.**

Remark: if line 2 had been `new int32_t[500]` instead of `[size]`, that allocation would be $500 = \Theta(1)$, and the whole function would use **$\Theta(1)$** auxiliary space.

4.10 Profiling and Performance Measurement Tools

4.10.1 Introduction to profiling

- Complexity analysis helps you **pick** the right algorithm; you still have to **implement** it correctly. A perfect analysis plus a faulty implementation = an inefficient solution.
- Profiling** = analyzing a program's resource usage **experimentally**, using tools called **profilers**.
- Profilers measure execution time, memory usage, CPU consumption, I/O activity → identify resource **bottlenecks** and slow / memory-heavy sections.
- Two tools used in this class: **perf** and **massif**.

4.10.2 Measuring runtime in Linux: the `time` command

```
time ./exec

real 0m2.059s
user 0m0.345s
sys 0m0.919s
```

- user time** = time spent by the program doing its own work.
- system time** = time spent by the OS on your behalf (e.g. reading/writing files).
- real (elapsed) time** = real-world wall-clock time between start and end.
- CPU utilization:**

$$\%CPU = (user + system) / elapsed$$

- Why isn't %CPU always 100%? The program does not own the CPU the whole time. Modern OSes run many processes and assign each a **time quantum** (usually milliseconds) before the CPU may be handed to another process. More simultaneous programs → lower %CPU.

4.10.3 perf

```
perf record --call-graph=dwarf -e cycles:u time ./exec
perf report
```

- Linux profiling tool that measures time spent in different parts of your code by **sampling**: it periodically interrupts the process and records its current state.
- Top of output shows the number of samples (e.g. 1K = 1,000). **More samples = more accurate** results.
- Each row is a function ("Symbol" column). The "**Children**" column = time spent in that function **plus all functions it calls**.
- You can highlight a row and press enter to **expand a parent into its child functions**.

perf example

Code with `call10/20/30/40` (each printing $65535 \times 1, \times 2, \times 3, \times 4$ lines), `function1()` calling `call20+call40`, `function2()` calling `call10+call30`.

Symbol	% of samples
function1()	60.90%
function2()	38.73%
call40()	39.91%
call30()	28.38%
call20()	20.92%
call10()	10.27%

`call40()` should be doing 40% of the work, and 39.91% matches → implementation looks correct.

Cautions when reading perf output

- A large percentage is **not automatically bad** if that function is expected to be expensive.
- A mismatch is the red flag: e.g. a $\Theta(\log(n))$ function taking a larger share than a $\Theta(n^2)$ function suggests an implementation error.
- Check the **sample count**. Small sample sizes give misleading results — run with a **larger input size** to collect more samples.

4.10.4 massif

- massif** is a **heap profiler** from **Valgrind** (a Linux instrumentation framework mainly used to detect memory issues; see chapter 6).
- It tracks heap allocations over time and reports **when and where memory consumption peaks**.

```
g++ -g3 ...           # compile with -g3, executable named msf
valgrind --tool=massif --massif-out-file=massif.out --time-unit=B ./msf
ms_print massif.out    # human-readable report (include any flags/input files the exe needs)
```

- Output starts with an ASCII graph of memory over time; each vertical bar is a **snapshot**.
- Example run: 50 snapshots (0-49); **detailed snapshots** at [5, 15, 24, 27 (peak), 37, 47] give per-line allocation breakdowns; the rest only give totals.
- Columns: `n`, `time(B)`, `total(B)`, `useful-heap(B)`, `extra-heap(B)`, `stacks(B)`.
- useful-heap** = bytes your program explicitly requested. **extra-heap** = overhead bytes added for metadata/memory management.

$$total = useful-heap + extra-heap$$

- `--time-unit=B` measures time in total allocation/deallocation events processed; use `--time-unit=ms` for real milliseconds instead.

Reading the example program's snapshots

```
1 int main() {
2   int32_t **a = new int32_t*[200000];
3   int32_t *b = new int32_t[1000000];
4   int32_t *c = new int32_t[500000];
5   delete[] b;
6   for (int32_t i = 0; i < 16; ++i) {
7     a[i] = new int32_t[25000];
8   } // for i
9   int32_t *d = new int32_t[750000];
10  delete[] c;
11  int32_t *e = new int32_t[750000];
12  for (int32_t i = 0; i < 16; ++i) {
13    delete[] a[i];
14  } // for i
15  int32_t *f = new int32_t[250000];
16  delete[] a; 17 delete[] f; 18 delete[] d; 19 delete[] e;
20 } // main()
```

- **Snapshot 5** — total 7,672,736 B (7,672,704 requested + 32 extra). Breakdown: line 3 = 4,000,000 B (1,000,000 ints × 4 B) = 52.13%; line 4 = 2,000,000 B (500,000 ints) = 26.07%; line 2 = 1,600,000 B (200,000 pointers × 8 B) = 20.85%.
- **Snapshot 15** — 4,572,800 B in use. Line 3's memory is gone (deleted on line 5) — visible as the 4,000,000 B drop between snapshots 5 and 6. Now line 4 = 43.74%, line 2 = 34.99%, line 7 = 19.68% (900,000 B). Each snapshot after 6 is one more iteration of the line-6 loop, adding 100,000 B (25,000 ints × 4 B). 900,000 B means the loop has run 9 times.
- **Snapshots 23/24** — allocation on line 9 and deallocation on line 10; array d is now the biggest contributor at 36.26%.
- **Snapshot 27 = peak** (9,272,864 B), right before line 12. d (line 9) and e (line 11) each contribute 32.35%.

Why the peak matters: assignments may be graded on **peak memory usage**, so the peak snapshot is the one to investigate when you are over the memory limit. Tip: look for **unnecessary copies of large objects**.

Example 4.20 — where was snapshot 37 taken?

Snapshots 28–37 drop by exactly 100,000 B each → we are inside the **deallocation loop on line 12**. Array a's contribution went from 1,600,000 B at peak to 600,000 B. Each iteration frees 100,000 B, so 1,000,000 B freed = **10 iterations** of the line-12 loop have completed.

Example 4.21 — where was snapshot 47 taken?

At snapshot 47 only 3,000,000 B remain from main(), all from array e (line 11) at 97.63%. So snapshot 47 was taken **after d was deallocated on line 18**. Snapshot 48 shows e freed; snapshot 49 shows everything cleaned up before the program ends.

Example 4.22 — solve for X

```
51 157,440 2,416 [-X-] 16 0
```

total = useful-heap + extra-heap → useful-heap = 2,416 − 16 = **2,400 bytes**.

Chapter takeaway: measure how runtime *scales*, not how long it runs. Assign each line a complexity, condense loops inside-out, add sequential pieces, drop coefficients and lower order terms. Watch for logs (multiply/divide updates), loop dependencies (inner bound depends on outer variable), and remember that a better complexity class does not guarantee a faster program.